

4

SECURITY ACCESS TOKENS



The *security access token*, or *token* for short, is at the heart of Windows security. The SRM uses tokens to represent identities, such as user accounts, and then grants or denies them access to resources. Windows represents tokens with `Token` kernel objects, which contain, at a minimum, the specific identity they represent, any security groups the identity belongs to, and the special privileges the identity has

been granted.

Like other kernel objects, tokens support `Query` and `Set` information system calls, which allow the user to inspect the properties of a token and set certain properties. Though less commonly used, some Win32 APIs also expose these `Set` and `Query` system calls: for example, `GetTokenInformation` and `SetTokenInformation`.

Let's start with an overview of the two main types of tokens you'll encounter when analyzing a Windows system's security: primary and impersonation tokens. We'll then detail many of the important properties a token contains. You'll need to understand these before we can discuss access checking in [Chapter 7](#).

Primary Tokens

Every process has an assigned token that describes its identity for any resource access operation. When the SRM performs an access check, it will query the process's token and use it to determine what kind of access to grant. When a token is used for a process, it's called a *primary token*.

You can open a process's token using the `NtOpenProcessToken` system call, which will return a handle that you can use to query token information. Because the `Token` object is a securable resource, the caller needs to pass an access check to get the handle. Note that you also need a handle to the process with `QueryLimitedInformation` access to be able to query the token.

When opening a `Token` object, you can request the following access rights:

`AssignPrimary` Assigns the `Token` object as a primary token

- Duplicate** Duplicates the Token object
- Impersonate** Impersonates the Token object
- Query** Queries the properties of the Token object, such as its groups and privileges
- QuerySource** Queries the source of the Token object
- AdjustPrivileges** Adjusts a Token object's privilege list
- AdjustGroups** Adjusts a Token object's group list
- AdjustDefault** Adjusts properties of a Token object not covered by the other access rights
- AdjustSessionId** Adjusts the Token object's session ID

You can see a list of accessible processes and their tokens by running the PowerShell command `Show-NtToken -All`. This should open the Token Viewer application, as shown in [Figure 4-1](#).

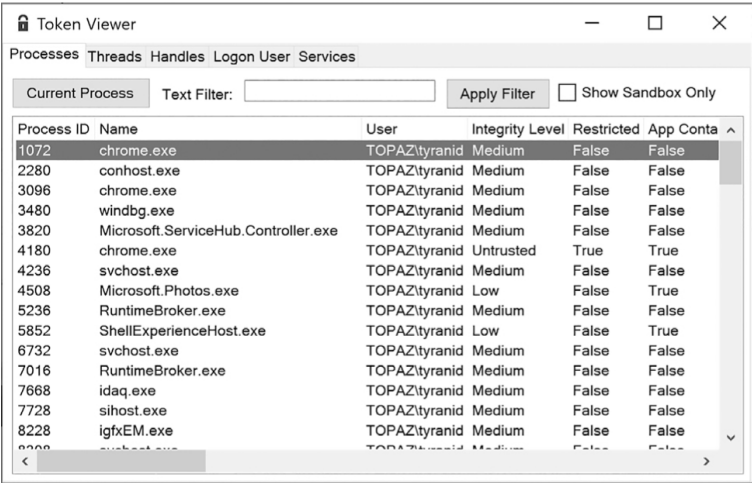


Figure 4-1: The Token Viewer lists all accessible processes and their tokens.

The list view provides only a simple overview of the available tokens. If you want to see more information, double-click one of the process entries to bring up a detailed view of the token, as shown in [Figure 4-2](#).

Current - User GRAPHITE\user - TokenId 00000000-0055...

Main Details Groups Privileges Default Dacl Misc Operations Token Source Security

Token

User: GRAPHITE\user

User SID: S-1-5-21-2321330777-2221220761-3053509190-1002

Token Type: Primary

Impersonation Level: N/A

Token ID: 00000000-0055BA64

Authentication ID: 00000000-0041B6B3

Origin Login ID: 00000000-000003E7

Modified ID: 00000000-0054BB5C

Integrity Level: Medium Set Integrity Level

Session ID: 4

Elevation Type: Limited Linked Token

Is Elevated: False

Source

Name: User32

Id: 00000000-0041B66F

Figure 4-2: The detailed view for a process's Token object

Let's highlight a few important pieces of information in this view. At the top are the user's name and SID. The `Token` object stores only the SID, but the token view will display the name if it's available. The next field indicates the token's type. As we're inspecting a primary token, the type is set to `Primary`. The impersonation level (below this) is used only for impersonation tokens, which we'll discuss in the next section. It's not needed for primary tokens, so it's set to `N/A`.

In the middle of the dialog is a list of four 64-bit integer identifiers:

Token ID A unique value assigned when the `Token` object was created

Authentication ID A value that indicates the logon session the token belongs to

Origin Login ID The authentication identifier of the parent logon session

Modified ID A unique value that is updated when certain token values are modified

LSASS creates a *logon session* when a user authenticates to a Windows machine. The logon session tracks authentication-related resources for a user; for example, it stores a copy of the user's credentials so that they can be reused. During the logon session creation process, the SRM generates a unique authentication identifier value that can be used to reference the session. Therefore, for a given logon session, all user tokens will have the same authentication identifier. If a user authenticates twice to the same machine, the SRM will generate different authentication identifiers.

The origin login identifier indicates who created the token's logon session. If you authenticate a different user on your desktop (by calling the `LogonUser` API with a username and password, for example), then the origin login identifier will serve as the calling token's authentication identifier. Notice that this field in [Figure 4-2](#) shows the value `00000000-000003E7`. This is one of four fixed authentication identifiers defined by the SRM, in this case indicating the *SYSTEM* logon session. [Table 4-1](#) shows the four fixed values, along with the SIDs for the user accounts associated with the sessions.

Table 4-1: Authentication Identifiers and User SIDs for Fixed Logon Sessions

Authentication identifier	User SID	Logon session username
00000000-000003E4	S-1-5-20	NT AUTHORITY\NETWORK SERVICE
00000000-000003E5	S-1-5-19	NT AUTHORITY\LOCAL SERVICE
00000000-000003E6	S-1-5-7	NT AUTHORITY\ANONYMOUS LOGON
00000000-000003E7	S-1-5-18	NT AUTHORITY\SYSTEM

After the identifiers in the detail view is a field indicating the *integrity level* of the token. Windows Vista first added the integrity level to implement a simple *mandatory access control* mechanism, whereby system-wide policies enforce access to resources, rather than allowing an individual resource to specify its access. We'll discuss integrity levels in "Token Groups" on page 109.

This is followed by the session ID, a number assigned to the console session the process is attached to. Even though the console session is a property of the process, the value is specified in the process's token.

LOCALLY UNIQUE IDENTIFIERS

I mentioned that a token's identifiers are 64-bit integers. Technically, they're *locally unique identifier (LUID)* structures containing two 32-bit values. LUIDs are a common system type, and the SRM uses them when it needs a unique value. For example, they're used to uniquely identify privilege values.

You can allocate your own LUID by calling the `NtAllocateLocallyUniqueId` system call or executing the `Get-NtLocallyUniqueId` PowerShell command. When you use a system call, Windows ensures it has a central authority for generating the next unique ID. This is important, as reusing a value might be catastrophic. For instance, if a LUID were reused as the authentication identifier for a token, it might overlap with one of the identifiers defined in [Table 4-1](#). This could trick

the system into thinking a more privileged user was accessing a resource, resulting in privilege escalation.

The Token Viewer GUI is great if you want to manually inspect a token's information. For programmatic access, you can open a `Token` object in PowerShell using the `Get-NtToken` command. Use the following to get the current process's token:

```
PS> $token = Get-NtToken
```

If you want to open the token for a specific process, you can use this command, replacing `<PID>` with the process ID of the target process:

```
PS> $token = Get-NtToken -ProcessId <PID>
```

The result of the `Get-NtToken` command is a `Token` object whose properties you can query. For example, you can display the token's user, as shown in [Listing 4-1](#).

```
PS> $token.User
Name              Attributes
----              -
GRAPHITE\user     None
```

Listing 4-1: Displaying the user via a Token object's properties

Use the `Format-NtToken` command to output basic information to the console, as shown in [Listing 4-2](#).

```
PS> Format-NtToken $token -All
USER INFORMATION
-----
Name              Attributes
----              -
GRAPHITE\user     None

GROUP SID INFORMATION
-----
Name              Attributes
----              -
GRAPHITE\None     Mandatory, EnabledByDefault
Everyone          Mandatory, EnabledByDefault
--snip--
```

Listing 4-2: Displaying properties of a token using Format-NtToken

You can pass the opened `Token` object to `Show-NtToken` to display the same GUI shown in [Figure 4-2](#).

Impersonation Tokens

The other type of token you'll encounter is the *impersonation token*.

Impersonation tokens are most important for system services, as they allow a process with one identity to temporarily impersonate another identity for the purposes of an access check. For example, a service might need to open a file belonging to another user while performing some operation. By allowing that service to impersonate the calling user, the system grants it access to the file, even if the service couldn't open the file directly.

Impersonation tokens are assigned to threads, not processes. This means that only the code running in that thread will take on the impersonated identity. There are three ways an impersonation token can be assigned to a thread:

- By explicitly granting a `Token object Impersonate` access and a `Thread object SetThreadToken` access
- By explicitly granting a `Thread object DirectImpersonation` access
- Implicitly, by impersonating an RPC request

You're most likely to encounter implicit token assignment, as it's the most common case for system services, which expose RPC mechanisms. For example, if a service creates a named pipe server, it can impersonate clients that connect to the pipe using the `ImpersonateNamedPipe` API. When a call is made on the named pipe, the kernel captures an *impersonation context* based on the calling thread and process. This impersonation context is used to assign an impersonation token to the thread that calls `ImpersonateNamedPipe`. The impersonation context can be based on either an existing impersonation token on the thread or a copy of the process's primary token.

Security Quality of Service

What if you don't want to give the service the ability to impersonate your identity? The SRM supports a feature called *Security Quality of Service* (SQoS) that enables you to control this. When you open a named pipe using the filesystem APIs, you can pass a `SECURITY_QUALITY_OF_SERVICE` structure in the `SecurityQualityOfService` field of the `OBJECT_ATTRIBUTES` structure. The SQoS structure contains three configuration values: the impersonation level, the context tracking mode, and the effective token mode.

The *impersonation level* in the SQoS is the most important field for controlling what a service can do with your identity. It defines the level of access granted to the service when it implicitly impersonates the caller. The level can be one of four values, in ascending order of privilege:

1. **Anonymous:** Prevents the service from opening the `Token` object and querying the user's identity. This is the lowest level; only a limited set of services would function if the caller specified this level.
2. **Identification:** Allows the service to open the `Token` object and query the user's identity, groups, and privileges. However, the thread cannot

open any secured resources while impersonating the user.

3. **Impersonation:** Allows the service to fully exercise the user's identity on the local system. The service can open local resources secured by the user and manipulate them. It can also access remote resources for the user if the user has locally authenticated to the system. However, if the user authenticated over a network connection, such as via the Server Message Block (SMB) protocol, then the service can't use the `Token` object to access remote resources.

4. **Delegation:** Enables the service to open all local and remote resources as if they were the user. This is the highest level. To access a remote resource from network-authenticated users, however, it's not enough to have this impersonation level. The Windows domain must also be configured to allow it. We'll discuss this impersonation level more in [Chapter 14](#), on Kerberos authentication.

You can specify the impersonation level in the SQoS either when calling a service or when creating a copy of an existing token. To restrict what a service can do, specify the `Identification` or `Anonymous` level. This will prevent the service from accessing any resources, although at the `Identification` level the service will still be able to access the token and perform operations on the caller's behalf.

Let's run a test using the `Invoke-NtToken` PowerShell command. In [Listing 4-3](#), we impersonate a token at two different levels and attempt to execute a script that opens a secured resource. We specify the impersonation level using the `ImpersonationLevel` property.

```
PS> $token = Get-NtToken
PS> Invoke-NtToken $token {
    Get-NtDirectory -Path "\"
} -ImpersonationLevel Impersonation
Name NtTypeName
----
Directory

PS> Invoke-NtToken $token {
    Get-NtDirectory -Path "\"
} -ImpersonationLevel Identification
Get-NtDirectory : (0xC00000A5) - A specified impersonation level is invalid.
--snip--
```

Listing 4-3: Impersonating a token at different levels and opening a secured resource

The first command we execute gets a handle to the current process's primary token. We then call `Invoke-NtToken` to impersonate the token at the `Impersonation` level and run a script that calls `Get-NtDirectory` to open the root OMNS directory. The open operation succeeds, and we print the directory object to the console.

We then attempt to repeat the operation at the `Identification` level, but this time we receive a `STATUS_BAD_IMPERSONATION_LEVEL` error. (If you see this

error when developing an application or using the system, now you'll know the reason for it!) Note that the open operation doesn't return an "access denied" error, because the SRM doesn't get far enough to check whether the impersonated user can access the resource.

ANONYMOUS USERS

Specifying the Anonymous impersonation level is not the same as running as the *ANONYMOUS LOGON* user referenced in [Table 4-1](#). It's possible to run with an anonymous user identity and be granted access to a resource by an access check, but an Anonymous-level token cannot pass any access check, regardless of how the resource's security is configured.

The kernel implements the `NtImpersonateAnonymousToken` system call, which will impersonate the anonymous user on a specified thread. You can also access the anonymous user token using `Get-NtToken`:

```
PS> Get-NtToken -Anonymous | Format-NtToken
NT AUTHORITY\ANONYMOUS LOGON
```

The other two fields in the SQoS are used less frequently, but they're still important. The *context tracking mode* determines whether to statically capture the user's identity when a connection is made to the service. If the identity is not statically captured and the caller then impersonates another user before calling the service, the new impersonated identity will become available to the service, not to the process identity. Note that the impersonated identity can be passed to the service only if it's at the Impersonation or Delegation level. If the impersonated token is at the Identification or Anonymous level, the SRM generates a security error and rejects the impersonation operation.

Effective token mode changes the token passed to the server in a different way. It's possible to disable groups and privileges before making a call, and if effective token mode is disabled, the server can reenable those groups and privileges and use them. However, if effective token mode is enabled, the SRM will strip out the groups and privileges so that the server can't reenable them or use them.

By default, if no SQoS structure is specified when opening the interprocess communication (IPC) channel, the caller's level is Impersonation with static tracking and a noneffective token. If an impersonation context is captured and the caller is already impersonating, then the impersonation level of the thread token must be greater than or equal to the Impersonation level; otherwise, the capture will fail. This is enforced even if the SQoS requests the Identification level. This is an important security feature; it prevents a caller at the Identification level or below from calling over an RPC channel and pretending to be another user.

NOTE

I've described how SQoS is specified at the native system call level, as the `SECURITY_QUALITY_OF_SERVICE` structure is not exposed through the Win32 APIs directly. Instead, it's usually specified using additional flags; for example, `CreateFile` exposes SQoS by specifying the `SECURITY_SQOS_PRESENT` flag.

Explicit Token Impersonation

There are two ways to impersonate a token explicitly. If you have an impersonation `Token` object handle with `Impersonate` access, you can assign it to a thread using the `NtSetInformationThread` system call and the `ThreadImpersonationToken` information class.

If instead you have a thread you want to impersonate with `Direct Impersonation` access, you can use the other mechanism. With the handle to a source thread, you can call the `NtImpersonateThread` system call and assign an impersonation token to another thread. Using `NtImpersonateThread` is a mix between explicit and implicit impersonation. The kernel will capture an impersonation context as if the source thread has called over a named pipe. You can even specify the SQoS structure to the system call.

You might be thinking that impersonation opens a giant security backdoor. If I set up my own named pipe and convince a privileged process to connect to me, and the caller doesn't set SQoS to limit access, can't I gain elevated privileges? We'll come back to how this is prevented in "Token Assignment" on page 133.

Converting Between Token Types

You can convert between the two token types using duplication. When you duplicate a token, the kernel creates a new `Token` object and makes a deep copy of all the object's properties. While the token is duplicating, you can change its type.

This duplication operation differs from the handle duplication we discussed in [Chapter 3](#), as duplicating a handle to a token would merely create a new handle pointing to the same `Token` object. To duplicate the actual `Token` object, you need to have `Duplicate` access rights on the handle.

You can then use either the `NtDuplicateToken` system call or the `Copy -NtToken` PowerShell command to duplicate the token. For example, to create an impersonation token at the Delegation level based on an existing token, use the script in [Listing 4-4](#).

```
PS> $imp_token = Copy-NtToken -Token $token -ImpersonationLevel Delegation
PS> $imp_token.ImpersonationLevel
Delegation

PS> $imp_token.TokenType
Impersonation
```

Listing 4-4: Duplicating a token to create an impersonation token

You can convert the impersonation token back to a primary token by using `Copy-NtToken` again, as shown in [Listing 4-5](#).

```
PS> $pri_token = Copy-NtToken -Token $imp_token -Primary
PS> $pri_token.TokenType
Primary

PS> $pri_token.ImpersonationLevel
Delegation
```

Listing 4-5: Converting an impersonation token to a primary token

Note something interesting in the output: the new primary token has the same impersonation level as the original token. This is because the SRM considers only the `TokenType` property; if the token is a primary token, the impersonation level is ignored.

Seeing as we can convert an impersonation token back to a primary token, you might be wondering: Could we convert an Identification-level or Anonymous-level token back to a primary token, create a new process, and bypass the SQoS settings? Let's try it in [Listing 4-6](#).

```
PS> $imp_token = Copy-NtToken -Token $token -ImpersonationLevel Identification
PS> $pri_token = Copy-NtToken -Token $imp_token -Primary
Exception: "(0xC00000A5) - A specified impersonation level is invalid."
```

Listing 4-6: Duplicating an Identification-level token back to a primary token

This listing shows that we can't duplicate an Identification-level token back to a primary token. The second line causes an exception, because the operation would break a security guarantee of the SRM (specifically, that the SQoS allows the caller to control how its identity is used).

A final note: if you're opening a token using `Get-NtToken`, you can perform the duplication operation in one step by specifying the `Duplicate` parameter.

Pseudo Token Handles

To access a token, you must open a handle to the `Token` object, then remember to close the handle after use. Windows 10 introduced three *pseudo handles* that allow you to query token information without opening a full handle to a kernel object. Here are those three handles, with their handle values in parentheses:

Primary (-4) The primary token for the current process

Impersonation (-5) The impersonation token for the current thread; fails if the thread is not impersonating

Effective (-6) The impersonation token for the current thread, if it is impersonating; otherwise, the primary token

Unlike the current process and current thread pseudo handles, you can't duplicate these token handles; you can use them for certain limited uses only, such as querying information or performing access checks. The `Get-NtToken` command can return these handles if you specify the `Pseudo` parameter, as shown in [Listing 4-7](#).

```
PS> Invoke-NtToken -Anonymous {Get-NtToken -Pseudo -Primary | Get-NtTokenSid}
Name                               Sid
----                               -
GRAPHITE\user                      S-1-4-21-2318445812-3516008893-216915059-1002 ❶

PS> Invoke-NtToken -Anonymous {Get-NtToken -Pseudo -Impersonation | Get-NtTokenSid}
Name                               Sid
----                               -
NT AUTHORITY\ANONYMOUS LOGON       S-1-4-7 ❷

PS> Invoke-NtToken -Anonymous {Get-NtToken -Pseudo -Effective | Get-NtTokenSid}
Name                               Sid
----                               -
NT AUTHORITY\ANONYMOUS LOGON       S-1-4-7 ❸

PS> Invoke-NtToken -Anonymous {Get-NtToken -Pseudo -Effective} | Get-NtTokenSid
Name                               Sid
----                               -
GRAPHITE\user                      S-1-4-21-2318445812-3516008893-216915059-1002 ❹
```

Listing 4-7: Querying pseudo tokens

Here, we query the three types of pseudo tokens while impersonating the anonymous user. The first command queries the primary token and extracts its user SID ❶. The next command queries the impersonation token, which returns the anonymous user's SID ❷. We then query the effective token, which, as we're impersonating the anonymous user, also returns the anonymous user's SID ❸. Finally, we query the effective token again, this time waiting until after the script block has executed to extract the user SID. This operation returns the primary token's user SID ❹, demonstrating that the pseudo token is context sensitive.

Token Groups

If administrators had to secure every resource for each possible user, identity security would become too unwieldy to manage. *Groups* allow users to share a broader security identity. Most of the access control operations on Windows grant access to groups rather than individual users.

From the SRM's perspective, a group is just another SID that could potentially define access to a resource. We can display the groups in the PowerShell console using the `Get-NtTokenGroup` command, as shown in [Listing 4-8](#).

```
PS> Get-NtTokenGroup $token
Name                               Attributes
----                               -
GRAPHITE\None                      Mandatory, EnabledByDefault, Enabled
```

Everyone	Mandatory, EnabledByDefault, Enabled
BUILTIN\Users	Mandatory, EnabledByDefault, Enabled
BUILTIN\Performance Log Users	Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\INTERACTIVE	Mandatory, EnabledByDefault, Enabled
--snip--	

Listing 4-8: Querying the current token's groups

We can also use `Get-NtTokenGroup` to filter for specific attribute flags by specifying the `Attributes` parameter. [Table 4-2](#) shows the possible attribute flags we can pass to the command.

Table 4-2: Group Attributes in SDK and PowerShell Format

SDK attribute name	PowerShell attribute name
SE_GROUP_ENABLED	Enabled
SE_GROUP_ENABLED_BY_DEFAULT	EnabledByDefault
SE_GROUP_MANDATORY	Mandatory
SE_GROUP_LOGON_ID	LogonId
SE_GROUP_OWNER	Owner
SE_GROUP_USE_FOR_DENY_ONLY	UseForDenyOnly
SE_GROUP_INTEGRITY	Integrity
SE_GROUP_INTEGRITY_ENABLED	IntegrityEnabled
SE_GROUP_RESOURCE	Resource

The following sections describe what each of these flags means.

Enabled, EnabledByDefault, and Mandatory

The most important flag is `Enabled`. When it's set, the SRM considers the group during the access check process; otherwise, it will ignore the group. Any group with the `EnabledByDefault` attribute set is automatically enabled.

It's possible to disable a group (excluding it from the access check process) using the `NtAdjustGroupsToken` system call if you have `AdjustGroups` access on the token handle; the `Set-NtTokenGroup` PowerShell command exposes this system call. However, you can't disable groups that have the `Mandatory` flag set. This flag is set for all groups in a normal user's token, but certain system tokens have nonmandatory groups. If a group is disabled when you pass an impersonation token over RPC and the effective token mode flag is set in the `SQoS`, the impersonation token will delete the group.

LogonId

The `LogonId` flag identifies any SID that is granted to all tokens on the same desktop. For example, if you run a process as a different user using the `runas` utility, the new process's token will have the same logon SID as the caller, even though it's a different identity. This behavior allows the SRM to grant access to session-specific resources, such as the session object directory. The SID is always in the format `S-1-4-4-X-Y`, where `X` and `Y` are the two 32-bit values of the LUID that was allocated when the authentication session was created. We'll come back to the logon SID and where it applies in the next chapter.

Owner

All securable resources on the system belong to either a group SID or a user SID. Tokens have an `Owner` property that contains a SID to use as the default owner when creating a resource. The SRM allows only a specific set of the users' SIDs to be specified in the `Owner` property: either the user's SID or any group SID that is marked with the `Owner` flag.

You can get or set the token's current `Owner` property using the `Get -NtTokenSid` or `Set-NtTokenSid` command. For example, in [Listing 4-9](#) we get the owner SID from the current token, then attempt to set the owner.

```
PS> Get-NtTokenSid $token -Owner
Name          Sid
----          ---
GRAPHITE\user S-1-4-21-818064984-378290696-2985406761-1002

PS> Set-NtTokenSid -Owner -Sid "S-1-2-3-4"
Exception setting "Owner": "(0xC000005A) - Indicates a particular
Security ID may not be assigned as the owner of an object."
```

Listing 4-9: Getting and setting the token's owner SID

In this case, our attempt to set the `Owner` property to the SID `S-1-2-3-4` fails with an exception, as this isn't our current user SID or in our list of groups.

UseForDenyOnly

The SRM's access check either allows or denies access to a SID. But when a SID is disabled, it will no longer participate in allow or deny checks, which can result in incorrect access checking.

Let's consider a simple example. Imagine there are two groups, *Employee* and *Remote Access*. A user creates a document that they want all employees to be able to read except for those remotely accessing the system, as the content of the document is sensitive and the user doesn't want it to leak. The document is configured to grant all members of the *Employee* group access but to deny access to users in the *Remote Access* group.

Now imagine that a user belonging to both of those groups could disable a group when accessing a resource. They could simply disable *Remote*

Access to be granted access to the document based on their membership in the *Employee* group, trivially circumventing the access restrictions.

For this reason, a user will rarely be allowed to disable groups. However, in certain cases, such as sandboxing, you'll want to be able to disable a group so that it can't be used to access a resource. The `UseForDenyOnly` flag solves this problem. When a SID is marked with this flag, it won't be considered when checking for allow access but will still be considered in deny access checks. A user can mark their own groups as `UseForDenyOnly` by filtering their token and using it to create a new process. We'll discuss token filtering when we consider restricted tokens in "Sandbox Tokens" on page 117.

Integrity and IntegrityEnabled

The `Integrity` and `IntegrityEnabled` attribute flags indicate that a SID represents the token's integrity level and is enabled. Group SIDs marked with the `Integrity` attribute flag store this integrity level as a 32-bit number in their final RID. The RID can be any arbitrary value; however, there are seven predefined levels in the SDK, as shown in [Table 4-3](#). Only the first six are in common use and accessible from a user process. To indicate an integrity SID the SRM uses the `MandatoryLabel` security authority (which has the value 16).

Table 4-3: Predefined Integrity Level Values

Integrity level	SDK name	PowerShell name
0	SECURITY_MANDATORY_UNTRUSTED_RID	Untrusted
4096	SECURITY_MANDATORY_LOW_RID	Low
8192	SECURITY_MANDATORY_MEDIUM_RID	Medium
8448	SECURITY_MANDATORY_MEDIUM_PLUS_RID	MediumPlus
12288	SECURITY_MANDATORY_HIGH_RID	High
16384	SECURITY_MANDATORY_SYSTEM_RID	System
20480	SECURITY_MANDATORY_PROTECTED_PROCESS_RID	ProtectedProcess

The default level for a user is `Medium`. Administrators are usually assigned `High`, and services are assigned `System`. We can query a token's integrity SID using `Get-NtTokenSid`, as shown in [Listing 4-10](#).

```
PS> Get-NtTokenSid $token -Integrity
Name                               Sid
----                               ---
Mandatory Label\Medium Mandatory Level S-1-16-8192
```

Listing 4-10: Getting a token's integrity level SID

We can also set a new token integrity level, provided it's less than or equal to the current value. It's possible to increase the level too, but this requires special privileges and having `SeTcbPrivilege` enabled.

While you can set the entire SID, it's usually more convenient to set just the value. For example, the script in [Listing 4-11](#) will set a token's integrity level to the `Low` level.

```
PS> Set-NtTokenIntegrityLevel Low -Token $token
PS> Get-NtTokenSid $token -Integrity
Name                               Sid
----                               -
Mandatory Label\Low Mandatory Level S-1-16-4096
```

Listing 4-11: Setting the token integrity level to Low

If you run this script, you might find that you start to get errors in your PowerShell console due to blocked file access. We'll discuss why file access is blocked when we cover Mandatory Integrity Control in [Chapter 7](#).

Resource

The final attribute flag deserves only a passing mention. The `Resource` attribute flag indicates that the group SID is a *domain local SID*. We'll come back to this SID type in [Chapter 10](#).

Device Groups

A token can also have a separate list of *device groups*. These group SIDs are added when a user authenticates to a server over a network in an enterprise environment, as shown in [Listing 4-12](#).

```
PS> Get-NtTokenGroup -Device -Token $token
Name                               Attributes
----                               -
BUILTIN\Users                      Mandatory, EnabledByDefault, Enabled
AD\CLIENT1$                        Mandatory, EnabledByDefault, Enabled
AD\Domain Computers                Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\Claims Value          Mandatory, EnabledByDefault, Enabled
--snip--
```

Listing 4-12: Displaying device groups using `Get-NtTokenGroup`

You can query the groups on the token by using `Get-NtTokenGroup` and passing the `Device` parameter.

Privileges

Groups allow system administrators to control a user's access to specific resources. *Privileges*, in contrast, are granted to a user to allow them to short-circuit certain security checks for all types of resources, such as by

bypassing an access check. A privilege can also apply to certain privileged actions, like changing the system's clock. You can view a token's privileges in the console using `Get-NtTokenPrivilege` ([Listing 4-13](#)).

```
PS> Get-NtTokenPrivilege $token
```

Name	Luid	Enabled
----	----	-----
SeShutdownPrivilege	00000000-00000013	False
SeChangeNotifyPrivilege	00000000-00000017	True
SeUndockPrivilege	00000000-00000019	False
SeIncreaseWorkingSetPrivilege	00000000-00000021	False
SeTimeZonePrivilege	00000000-00000022	False

Listing 4-13: Listing token privileges

The output is split into three columns. The first column is the privilege's common name. As with SIDs, the SRM does not use this name directly; instead, it uses the privilege's LUID value, which we can see in the second column. The last column indicates whether the privilege is currently enabled. Privileges can be in an enabled or disabled state.

Any check for a privilege should make sure that the privilege is enabled and not just present. In certain circumstances, such as sandboxing, a token might have a privilege listed, but the sandbox restrictions might prevent it from being marked as enabled. The `Enabled` flag is really a set of attribute flags, like the attributes for the group SIDs. We can view these attributes by formatting the output of `Get-NtTokenPrivilege` as a list ([Listing 4-14](#)).

```
PS> Get-NtTokenPrivilege $token -Privileges SeChangeNotifyPrivilege | Format-List
```

Name	: SeChangeNotifyPrivilege
Luid	: 00000000-00000017
Attributes	: EnabledByDefault, Enabled
Enabled	: True
DisplayName	: Bypass traverse checking

Listing 4-14: Displaying all properties of the SeChangeNotifyPrivilege privilege

In the output, we can now see the attributes, which include both `Enabled` and `EnabledByDefault`. The `EnabledByDefault` attribute specifies whether the default state of the privilege is to be enabled. We also now see an additional `DisplayName` property, used to provide additional information to a user.

To modify the state of a token's privileges, you need `AdjustPrivileges` access on the token handle; then you can use the `NtAdjustPrivilegesToken` system call to adjust the attributes and enable or disable a privilege. The `Enable-NtTokenPrivilege` and `Disable-NtTokenPrivilege` PowerShell commands expose this system call, as shown in [Listing 4-15](#).

```
PS> Enable-NtTokenPrivilege SeTimeZonePrivilege -Token $token -PassThru
```

Name	Luid	Enabled
----	----	-----
SeTimeZonePrivilege	00000000-00000022	True

```
PS> Disable-NtTokenPrivilege SeTimeZonePrivilege -Token $token -PassThru
```

Name	Luid	Enabled
----	----	-----
SeTimeZonePrivilege	00000000-00000022	False

Listing 4-15: Enabling and disabling the SeTimeZonePrivilege privilege

Using the `NtAdjustPrivilegesToken` API, it's also possible to remove a privilege entirely by specifying the `Remove` attribute, which you can accomplish with the `Remove-NtTokenPrivilege` PowerShell command. Removing a privilege ensures that the token can never use it again. If you only disable the privilege, then it could be reenabled inadvertently. [Listing 4-16](#) shows how to remove a privilege.

```
PS> Get-NtTokenPrivilege $token -Privileges SeTimeZonePrivilege
```

Name	Luid	Enabled
----	----	-----
SeTimeZonePrivilege	00000000-00000022	False

```
PS> Remove-NtTokenPrivilege SeTimeZonePrivilege -Token $token
PS> Get-NtTokenPrivilege $token -Privileges SeTimeZonePrivilege
WARNING: Couldn't get privilege SeTimeZonePrivilege
```

Listing 4-16: Removing a privilege from a token

To check privileges, a user application can call the `NtPrivilegeCheck` system call, while kernel code can call the `SePrivilegeCheck` API. You might be wondering whether you can just manually test whether a privilege is enabled rather than using a dedicated system call. In this instance, yes; however, it's always worth using system facilities where possible in case you make a mistake in your implementation or haven't considered some edge case. The `Test-NtTokenPrivilege` PowerShell command wraps the system call, as shown in [Listing 4-17](#).

```
PS> Enable-NtTokenPrivilege SeChangeNotifyPrivilege
PS> Disable-NtTokenPrivilege SeTimeZonePrivilege
PS> Test-NtTokenPrivilege SeChangeNotifyPrivilege
True

PS> Test-NtTokenPrivilege SeTimeZonePrivilege, SeChangeNotifyPrivilege -All
False

PS> Test-NtTokenPrivilege SeTimeZonePrivilege, SeChangeNotifyPrivilege
-All -PassResult
```

EnabledPrivileges	AllPrivilegesHeld
-----	-----
{SeChangeNotifyPrivilege}	False

Listing 4-17: Performing privilege checks

This listing demonstrates some example privilege checks using `Test-NtTokenPrivilege`. We start by enabling `SeChangeNotifyPrivilege` and disabling `SeTimeZonePrivilege`. These are common privileges granted to all users, but

you might need to change the example if your token doesn't have them. We then test for just `SeChangeNotifyPrivilege`; it's enabled, so this test returns `True`. Next, we check for both `SeTimeZonePrivilege` and `SeChangeNotifyPrivilege`; we can see that we don't have all the privileges, so `Test-NtTokenPrivilege` returns `False`. Finally, we run the same command but specify the `-PassResult` option to return the full check result. We can see in the `EnabledPrivileges` column that only `SeChangeNotifyPrivilege` is enabled.

The following are some of the privileges available on the system:

`SeChangeNotifyPrivilege` This privilege's name is misleading. It allows a user to receive notifications of changes to the filesystem or registry, but it's also used to bypass traversal checking. We'll discuss traversal checking in [Chapter 8](#).

`SeAssignPrimaryTokenPrivilege` and `SeImpersonatePrivilege` These privileges allow the user to bypass the assigning primary token and impersonation checks, respectively. Unlike most privileges in this list, these must be enabled on the current process's primary token, not on an impersonation token.

`SeBackupPrivilege` and `SeRestorePrivilege` These privileges allow the user to bypass the access check when opening specific resources, like files or registry keys. This lets the user back up and restore resources without needing to be granted access to them explicitly. These privileges have also been repurposed for other uses: for example, the restore privilege allows a user to load arbitrary registry hives.

`SeSecurityPrivilege` and `SeAuditPrivilege` The first of these privileges allows a user to be granted the `AccessSystemSecurity` access right on a resource. This allows the user to modify the resource's auditing configuration. The `SeAuditPrivilege` privilege allows a user to generate arbitrary object audit messages from a user application. We'll discuss auditing in [Chapters 5, 6, and 9](#).

`SeCreateTokenPrivilege` This privilege should be given to only a very select group of users, as it grants the ability to craft arbitrary tokens using the `NtCreateToken` system call.

`SeDebugPrivilege` The name of this privilege implies that it's necessary for debugging processes. However, that's not really the case, as it's possible to debug a process without it. The privilege does allow the user to bypass any access check when opening a process or thread object.

`SeTcbPrivilege` The name of this privilege comes from *trusted computing base (TCB)*, a term used to refer to the privileged core of the Windows operating system, including the kernel. This is a catch-all for privileged operations not covered by a more specific privilege. For example, it allows users to bypass the check for increasing the integrity level of a token (up to the limit of the `System` level), but also to specify a fallback exception handler for a process, two operations that have little in common.

SeLoadDriverPrivilege We can load a new kernel driver through the `NtLoadDriver` system call, although it's more common to use the SCM. This privilege is required to successfully execute that system call. Note that having this privilege doesn't allow you to circumvent kernel driver checks such as code signing.

SeTakeOwnershipPrivilege and **SeRelabelPrivilege** These privileges have the same immediate effect: they allow a user to be granted `WriteOwner` access to a resource, even if the normal access control wouldn't allow it.

`SeTakeOwnershipPrivilege` allows a user to take ownership of a resource, as having `WriteOwner` is necessary for that purpose. `SeRelabelPrivilege` bypasses checks on the mandatory label of a resource; normally, you can only set a label to be equal to or lower than the caller's integrity level. Setting the mandatory label also requires `WriteOwner` access on a handle, as we'll see in [Chapter 6](#).

We'll look at specific examples of these privileges' uses in later chapters, when we discuss security descriptors and access checks. For now, let's turn to ways of restricting access through sandboxing.

Sandbox Tokens

In our connected world, we must process a lot of untrusted data. Attackers might craft data for malicious purposes, such as to exploit a security vulnerability in a web browser or a document reader. To counter this threat, Windows provides a method of restricting the resources a user can access by placing any processes of theirs that handle untrusted data into a sandbox. If the process is compromised, the attacker will have only a limited view of the system and won't be able to access the user's sensitive information. Windows implements sandboxes through three special token types: restricted tokens, write-restricted tokens, and lowbox tokens.

Restricted Tokens

The *restricted token* type is the oldest sandbox token in Windows. It was introduced as a feature in Windows 2000 but not used widely as a sandbox until the introduction of the Google Chrome web browser. Other browsers, such as Firefox, have since replicated Chrome's sandbox implementation, as have document readers such as Adobe Reader.

You can create a restricted token using the `NtFilterToken` system call or the `CreateRestrictedToken` Win32 API, each of which lets you specify a list of restricted SIDs to limit the resources the token will be permitted to access. The SIDs do not have to already be available in the token. For example, Chrome's most restrictive sandbox specifies the NULL SID (`s-1-0-0`) as the only restricted SID. The NULL SID is never granted to a token as a normal group.

Any access check must allow both the normal list of groups and the list of restricted SIDs; otherwise, the user will be denied access, as we'll discuss in detail in [Chapter 7](#). The `NtFilterToken` system call can also mark normal groups with the `UseForDenyOnly` attribute flag and delete privileges. We can

combine the ability to filter a token with restricted SIDs or use it on its own, to create a lesser-privileged token without more comprehensive sandboxing.

It's easy to build a restricted token that can't access any resources. Such a restriction produces a good sandbox but also makes it impossible to use the token as a process's primary token, as the process won't be able to start. This puts a serious limitation on how effective a sandbox using restricted tokens can be. **Listing 4-18** demonstrates how to create a restricted token and extract the results.

```
PS> $token = Get-NtToken -Filtered -RestrictedSids RC -SidsToDisable WD
-Flags DisableMaxPrivileges
PS> Get-NtTokenGroup $token -Attributes UseForDenyOnly
Name                               Attributes
----
Everyone                           UseForDenyOnly

PS> Get-NtTokenGroup $token -Restricted
Name                               Attributes
----
NT AUTHORITY\RESTRICTED            Mandatory, EnabledByDefault, Enabled

PS> Get-NtTokenPrivilege $token
Name                               Luid                               Enabled
----
SeChangeNotifyPrivilege           00000000-00000017                True

PS> $token.Restricted
True
```

Listing 4-18: Creating a restricted token and displaying groups and privileges

We start by creating a restricted token using the `Get-NtToken` command. We specify one restricted SID, `RC`, which maps to a special `NT AUTHORITY\RESTRICTED` SID that is commonly configured for system resources to permit read access. We also specify that we want to convert the `Everyone` group (`WD`) to `UseForDenyOnly`. Finally, we specify a flag to disable the maximum number of privileges.

Next, we display the properties of the token, starting with all normal groups, using the `UseForDenyOnly` attribute. The output shows that only the `Everyone` group has the flag set. We then display the restricted SIDs list, which shows the `NT AUTHORITY\RESTRICTED` SID.

After this, we display the privileges. Note that even though we've asked to disable the maximum privileges, the `SeChangeNotifyPrivilege` is still there. This privilege is not deleted, as it can become very difficult to access resources without it. If you really want to get rid of it, you can specify it explicitly to `NtFilterToken` or delete it after the token has been created.

Finally, we query the token property that indicates whether it's a restricted token.

INTERNET EXPLORER PROTECTED MODE

The first sandboxed web browser on Windows was Internet Explorer 7, introduced in Windows Vista. Internet Explorer 7 used the ability to lower the integrity level of a process's token to restrict the resources the browser could write to. Windows 8 ultimately replaced this simple sandbox, called *protected mode*, with a new type of token, the *lowbox* token, which we'll examine in "AppContainer and Lowbox Tokens" on page 120. The lowbox token provided greater isolation (called *enhanced protected mode*). It's interesting to note that Microsoft didn't use restricted tokens even though they had been available since Windows 2000.

Write-Restricted Tokens

A *write-restricted token* prevents write access to a resource but allows read and execute access. You can create a write-restricted token by passing the `WRITE_RESTRICTED` flag to `NtFilterToken`.

Windows XP SP2 introduced this token type to harden system services. It's much easier to use as a sandbox than a restricted token, as you don't need to worry about the token not being able to read critical resources such as DLLs. However, it creates a less useful sandbox. For example, if you can read files for a user, you might be able to steal their private information, such as passwords stored by a web browser, without needing to escape the sandbox.

For completeness, let's create a write-restricted token and view its properties ([Listing 4-19](#)).

```
PS> $token = Get-NtToken -Filtered -RestrictedSids WR -Flags WriteRestricted
PS> Get-NtTokenGroup $token -Restricted
Name                               Attributes
----                               -
NT AUTHORITY\WRITE RESTRICTED      Mandatory, EnabledByDefault, Enabled

PS> $token.Restricted
True

PS> $token.WriteRestricted
True
```

Listing 4-19: Creating a write-restricted token

We start by creating the token using the `Get-NtToken` command. We specify one restricted SID, `WR`, which maps to a special `NT AUTHORITY\WRITE RESTRICTED` SID that is equivalent to `NT AUTHORITY\RESTRICTED` but assigned to write access on specific system resources. We also specify the `WriteRestricted` flag to make this a write-restricted token rather than a normal restricted token.

Next, we display the token's properties. In the list of restricted SIDs, we see `NT AUTHORITY\WRITE RESTRICTED`. Displaying the `Restricted` property shows

that the token is considered restricted; however, we can see that it's also marked as `WriteRestricted`.

AppContainer and Lowbox Tokens

Windows 8 introduced the AppContainer sandbox to protect a new Windows application model. AppContainer implements its security using a *lowbox token*. You can create a lowbox token from an existing token with the `NtCreateLowBoxToken` system call. There is no direct equivalent Win32 API for this system call, but you can create an AppContainer process using the `CreateProcess` API. We won't go into more detail here on how to create a process using this API; instead, we'll focus only on the lowbox token.

When creating a lowbox token, you need to specify a package SID and a list of capability SIDs. Both SID types are issued by the *application package authority* (which has the value of 15). You can distinguish between package SIDs and capability SIDs by checking their first RIDs, which should be 2 and 3, respectively. The package SID works like the user's SID in the normal token, whereas the capability SIDs act like restricted SIDs. We'll leave the actual details of how these affect an access check for [Chapter 7](#).

Capability SIDs modify the access check process, but they can also mean something in isolation. For example, there are capabilities to allow network access that are handled specially by the Windows Firewall, even though that's not directly related to access checking. There are two types of capability SIDs:

Legacy A small set of predefined SIDs introduced in Windows 8

Named The RIDs are derived from a textual name

[Appendix B](#) contains a more comprehensive list of named capability SIDs. [Table 4-4](#) shows the legacy capabilities.

Table 4-4: Legacy Capability SIDs

Capability name	SID
Your internet connection	S-1-15-3-1
Your internet connection, including incoming connections from the internet	S-1-15-3-2
Your home or work networks	S-1-15-3-3
Your pictures library	S-1-15-3-4
Your videos library	S-1-15-3-5
Your music library	S-1-15-3-6

Capability name	SID
Your documents library	S-1-15-3-7
Your Windows credentials	S-1-15-3-8
Software and hardware certificates or a smart card	S-1-15-3-9
Removable storage	S-1-15-3-10
Your appointments	S-1-15-3-11
Your contacts	S-1-15-3-12
Internet Explorer	S-1-15-3-4096

We can use `Get-NtSid` to query for package and capability SIDs, as shown in [Listing 4-20](#).

```

PS> Get-NtSid -PackageName 'my_package' -ToSddl
❶ S-1-15-2-4047469452-4024960472-3786564613-914846661-3775852572-3870680127-2256146868

❷ PS> Get-NtSid -PackageName 'my_package' -RestrictedPackageName "CHILD" -ToSddl
S-1-15-2-4047469452-4024960472-3786564613-914846661-3775852572-3870680127-2256146868-951732652-15

❸ PS> Get-NtSid -KnownSid CapabilityInternetClient -ToSddl
S-1-15-3-1

❹ PS> Get-NtSid -CapabilityName registryRead -ToSddl
S-1-15-3-1024-1065365936-1281604716-3511738428-1654721687-432734479-3232135806-4053264122-3456934

❺ PS> Get-NtSid -CapabilityName registryRead -CapabilityGroup -ToSddl
S-1-5-32-1065365936-1281604716-3511738428-1654721687-432734479-3232135806-4053264122-3456934681

```

Listing 4-20: Creating package and capability SIDs

Here, we create two package SIDs and two capability SIDs. We generate the first package SID by specifying its name to `Get-NtSid` and receive the resulting SID ❶. This package SID is derived from the lowercase form of the name hashed with the SHA256 digest algorithm. The 256-bit digest is broken up into seven 32-bit chunks that act as the RIDs. The final 32-bit value of the digest is discarded.

Windows also supports a restricted package SID, which is designed to allow a package to create new secure child packages that can't interact with each other. The classic Edge web browser used this feature to separate internet- and intranet-facing children so that if one was compromised, it couldn't access data in the other. To create the child, we use the original package family name plus a child identifier ❷. The created SID extends

the original package SID with another four RIDs, as you can see in the output.

The first capability SID ❶ is a legacy capability for internet access. Note that the resulting SDDL SID has one additional RID value (1). The second SID is derived from a name, in this case `registryRead` ❷, which is used to allow read access to a group of system registry keys. As with the package SID, the named capability RIDs are generated from the SHA256 hash of the lowercase name. To differentiate between legacy and named capability SIDs, the second RID is set to 1024 followed by the SHA256 hash. You can generate your own capability SIDs using this method, although there's not much you can do with the capability unless some resource is configured to use it.

Windows also supports a *capability group*, a group SID that can be added to the normal list of groups ❸. A capability group sets the first RID to 32 and the rest of the RIDs to the same SHA256 hash that was derived from the capability name.

Now that we've got the SIDs, we can create a lowbox token as shown in

Listing 4-21.

```
❶ PS> $token = Get-NtToken -LowBox -PackageSid 'my_package'
    -CapabilitySid "registryRead", "S-1-15-3-1"
❷ PS> Get-NtTokenGroup $token -Capabilities | Select-Object Name
Name
----
NAMED CAPABILITIES\Registry Read
APPLICATION PACKAGE AUTHORITY\Your Internet connection

❸ PS> $package_sid = Get-NtTokenSid $token -Package -ToSddl
PS> $package_sid
S-1-15-2-4047469452-4024960472-3786564613-914846661-3775852572-3870680127 -2256146868

PS> Get-NtTokenIntegrityLevel $token
❹ Low

PS> $token.Close()
```

Listing 4-21: Creating a lowbox token and listing its properties

First we call `Get-NtToken`, passing it the package name (the SID as SDDL would also work) and the list of capabilities to assign to the lowbox token ❶. We can then query for the list of capabilities ❷. Notice that the names of the two capability SIDs are different: the SID derived from a name is prefixed with `NAMED CAPABILITIES`. There's no way of converting a named capability SID back to the name it was derived from; the PowerShell module must generate the name based on a large list of known capabilities. The second SID is a legacy SID, so LSASS can resolve it back to a name.

Next, we query the package SID ❸. As the package SID is derived from a name using SHA256, it's not possible to resolve it back to the package name. Again, the PowerShell module has a list of names that it can use to work out what the original name was.

A lowbox token is always set to the `Low` integrity level . In fact, if a privileged user changes the integrity level to `Medium` or above, all lowbox properties, such as package SIDs and capability SIDs, are removed, and the token reverts to a non-sandbox token.

We've covered making a user less privileged by converting their token into a sandbox token. We'll now go to the other side and look at what makes a user privileged enough to administrate the Windows system.

What Makes an Administrator User?

If you come from a Unix background, you'll know user ID 0 as the administrator account, or *root*. As *root*, you can access any resource and configure the system however you'd like. When you install Windows, the first account you configure will also be an administrator. However, unlike *root*, the account won't have a special SID that the system treats differently. So, what makes an administrator account on Windows?

The basic answer is that Windows is configured to give certain groups and privileges special access. Administrator access is inherently discretionary, meaning it's possible to be an administrator but still be locked out of resources; there is no real equivalent of a root account (although the *SYSTEM* user comes close).

Administrators generally have three characteristics. First, when you configure a user to be an administrator, you typically add them to the *BUILTIN\Administrators* group, then configure Windows to allow access to the group when performing an access check. For example, the system folders, such as *C:\Windows*, are configured to allow the group to create new files and directories.

Second, administrators are granted access to additional privileges, which effectively circumvent the system's security controls. For example, *SeDebugPrivilege* allows a user to get full access to any other process or thread on the system, no matter what security it has been assigned. With full access to a process, it's possible to inject code into it to gain the privileges of a different user.

Third, administrators typically run at the `High` integrity level, whereas system services run at the `System` level. By increasing the administrator's integrity level, we make it harder to accidentally leave administrator resources (especially processes and threads) accessible to non-administrators. Weak access control to resources is a common misconfiguration; however, if the resource is also marked with an integrity level above `Medium`, then non-administrator users won't be able to write to the resource.

A quick way to verify whether a token is an administrator is to check the `Elevated` property on the `Token` object. This property indicates whether the token has certain groups and available privileges found in a fixed list in the kernel. [Listing 4-22](#) shows an example for a non-administrator.

```
PS> $token = Get-NtToken
PS> $token.Elevated
False
```

Listing 4-22: The Elevated property for a non-administrator

If the token has one of the following privileges, it's automatically considered elevated:

- SeCreateTokenPrivilege
- SeTcbPrivilege
- SeTakeOwnershipPrivilege
- SeLoadDriverPrivilege
- SeBackupPrivilege
- SeRestorePrivilege
- SeDebugPrivilege
- SeImpersonatePrivilege
- SeRelabelPrivilege
- SeDelegateSessionUserImpersonatePrivilege

The privilege doesn't have to be enabled, just available in the token.

For elevated groups, the kernel doesn't have a fixed list of SIDs; instead, it inspects only the last RID of the SID. If the RID is set to one of the following values, then the SID is considered elevated: 114, 498, 512, 516, 517, 518, 519, 520, 521, 544, 547, 548, 549, 550, 551, 553, 554, 556, or 569. For example, the SID of the *BUILTIN\Administrators* group is S-1-4-32-544. As 544 is in this list, the SID is considered elevated. (Note that the SID S-1-1-2-3-4-544 would also be considered elevated, even though there is nothing special about it.)

HIGH INTEGRITY LEVEL DOESN'T EQUAL ADMINISTRATOR

It's a common misconception that if a token has a *High* integrity level, it's an administrator token. However, the *Elevated* property doesn't check a token's integrity level, just its privileges and groups. The *BUILTIN\Administrators* group would still function with a lower integrity level, allowing access to resources such as the Windows filesystem directory. The only restriction is that certain high-level privileges, such as *SeDebugPrivilege*, can't be enabled if the integrity level is less than *High*.

It is also possible for a non-administrator to run with a *High* integrity level, as in the case of UI access processes, which sometimes run at this integrity level but are not granted any special privileges or groups to make them an administrator.

User Account Control

I mentioned that when you install a new copy of Windows, the first user you create is always an administrator. It's important to configure the user in this way; otherwise, it would be impossible to modify the system and install new software.

However, prior to Windows Vista, this default behavior was a massive security liability, because average consumers would install the default account and likely never change it. This meant that most people used a full administrator account for everyday activities like surfing the web. If a malicious attacker were able to exploit a security issue in the user's browser, the attacker would get full control over the Windows machine. In the days prior to widespread sandboxing, this threat proved serious.

In Vista, Microsoft changed this default behavior by introducing *User Account Control (UAC)* and the split-token administrator. In this model, the default user remains an administrator; however, by default, all programs run with a token whose administrator groups and privileges have been removed. When a user needs to perform an administrative task, the system elevates a process to a full administrator and shows a prompt, like the one in [Figure 4-3](#), requesting the user's confirmation before continuing.

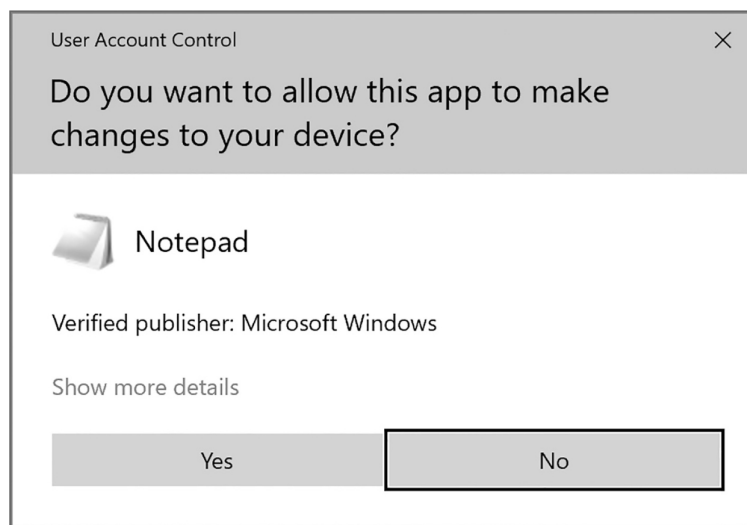


Figure 4-3: The UAC consent dialog for privilege elevation

To make using Windows easier for users, you can configure a program to force this elevation when it's started. A program's elevation property is stored in a manifest XML file embedded in the executable image. Run the example in [Listing 4-23](#) to get the manifest information for all the executables in the *System32* directory.

```
PS> ls C:\Windows\System32\*.exe | Get-Win32ModuleManifest
```

Name	UiAccess	AutoElevate	ExecutionLevel
aitstatic.exe	False	False	asInvoker
alg.exe	False	False	asInvoker
appidcertstorecheck.exe	False	False	asInvoker
appidpolicyconverter.exe	False	False	asInvoker
ApplicationFrameHost.exe	False	False	asInvoker
appverif.exe	False	False	highestAvailable

```
--snip--
```

Listing 4-23: Querying executable manifest information

If it's a special, Microsoft-approved program, the manifest can specify whether the program should be automatically, and silently, elevated (indicated by a `True` value in the `AutoElevate` column). The manifest also indicates whether the process can run with UI access, a topic we'll discuss later on page 129. There are three possible values for the `ExecutionLevel` column:

asInvoker Run the process as the user who created it. This is the default setting.

highestAvailable If the user is a split-token administrator, then force elevation to the administrator token. If not, then run as the user who created the process.

requireAdministrator Force elevation, whether the user is a split-token administrator or not. If the user is not an administrator, they'll be prompted for a password for an administrator account.

When something creates an executable with an elevated execution level, the shell calls the RPC method `RAILaunchAdminProcess`. This method checks the manifest and starts the elevation process, including showing the consent dialog. It's also possible to manually elevate any application by using the `ShellExecute` API, introduced in "Shell APIs" on page 89, and requesting the `runas` operation. PowerShell exposes this behavior using the `Start-Process` command, as shown here:

```
PS> Start-Process notepad -Verb runas
```

When you run this command, you should see the UAC prompt. If you click Yes in the consent dialog, *notepad.exe* should run as an administrator on the desktop.

Linked Tokens and Elevation Type

When an administrator authenticates to the desktop, the system tracks two tokens for the user:

Limited The unelevated token used for most running processes

Full The full administrator token, used only after elevation

The name *split-token administrator* comes from these two tokens, as the user's granted access is split between the limited and full tokens.

The `Token` object has a field used to link the two tokens together. The linked token can be queried using the `NtQueryInformationToken` system call and the `TokenLinkedToken` information class. In [Listing 4-24](#), we inspect some of the properties of these linked tokens using PowerShell.

```
❶ PS> Use-NtObject($token = Get-NtToken -Linked) {  
    Format-NtToken $token -Group -Privilege -Integrity -Information  
}  
GROUP SID INFORMATION
```

```

-----
Name                               Attributes
-----
❷ BUILTIN\Administrators           Mandatory, EnabledByDefault, Enabled, Owner
--snip--

PRIVILEGE INFORMATION
-----
Name                               Luid                Enabled
-----
SeIncreaseQuotaPrivilege           00000000-00000005  False
❸ SeSecurityPrivilege              00000000-00000008  False
SeTakeOwnershipPrivilege           00000000-00000009  False
--snip--

INTEGRITY LEVEL
-----
❹ High

TOKEN INFORMATION
-----
❺ Type                            : Impersonation
Imp Level                         : Identification
Auth ID                           : 00000000-0009361F
❻ Elevated                        : True
❼ Elevation Type: Full
Flags                             : NotLow

```

Listing 4-24: Displaying properties of the linked token

We access the linked token by passing the `Linked` parameter to `Get -NtToken`, ❶ and we format the token to display its groups, privileges, integrity level, and token information. In the list of groups, we can see the `BUILTIN\Administrators` group enabled ❷. We can also see that the list of privileges contains some high-level ones, such as `SeSecurityPrivilege` ❸. The combination of groups and privileges confirms that this is an administrator token.

The integrity level of the token is set to `High` ❹, which, as we discussed earlier, prevents the token from accidentally leaving sensitive resources accessible to non-administrator users. In the token information, we can see that there's an impersonation token at Identification level ❺. To get a token that can create a new process, the caller needs the `SeTcbPrivilege` privilege, which means only system services, such as the Application Information service, can get the token. Finally, we can see that the token is marked as elevated ❻ and that the token elevation type indicates this is the full token ❼. Let's compare this with the limited token ([Listing 4-25](#)).

```

❶ PS> Use-NtObject($token = Get-NtToken) {
    Format-NtToken $token -Group -Privilege -Integrity -Information
}
GROUP SID INFORMATION
-----
Name                               Attributes
-----
❷ BUILTIN\Administrators           UseForDenyOnly

```

```
--snip--

PRIVILEGE INFORMATION
-----
Name                               Luid                               Enabled
-----
❸ SeShutdownPrivilege             00000000-00000013 False
SeChangeNotifyPrivilege          00000000-00000017 True
SeUndockPrivilege                00000000-00000019 False
SeIncreaseWorkingSetPrivilege    00000000-00000021 False
SeTimeZonePrivilege             00000000-00000022 False

INTEGRITY LEVEL
-----
❹ Medium

TOKEN INFORMATION
-----
Type           : Primary
Auth ID        : 00000000-0009369B
❺ Elevated     : False
❻ Elevation Type: Limited
❼ Flags        : VirtualizeAllowed, IsFiltered, NotLow
```

Listing 4-25: Displaying properties of the limited token

We first get a handle to the current token and format it with the same formatting we used in [Listing 4-24](#) ❶. In the list of groups, we can see that *BUILTIN\Administrators* has been converted to a *UseForDenyOnly* group ❷. Any other group that would match the elevated RID check would be converted in the same way.

The list of privileges shows only five items ❸. These are the only five privileges that the limited token can have. The integrity level of the token is set to *Medium*, down from *High* in the full token ❹. In the token information, we can see that the token is not elevated ❺, and the elevation type indicates that this is the limited token ❻.

Finally, note that the flags contain the value *IsFiltered* ❼. This flag indicates the token has been filtered using the *NtFilterToken* system call. This is because, to create the limited token, LSASS will first create a new full token so that its authentication ID has a unique value. (If you compare the *Auth ID* values in Listings 4-24 and 4-25, you'll notice they are indeed different.) This allows the SRM to consider the two tokens to be in separate logon sessions. LSASS then passes the token to *NtFilterToken* with the *LuaToken* parameter flag to convert any elevated group to *UseForDenyOnly* and delete all privileges other than the five permitted ones. *NtFilterToken* does not drop the integrity level from *High* to *Medium*, though; that must be done separately. Lastly, LSASS calls *NtSetInformationToken* to link the two tokens together using the *TokenLinkedToken* information class.

There is a third type of elevation, *default*, used for any token not associated with a split-token administrator:

```
PS> Use-NtObject($token = Get-NtToken -Anonymous) {$token.ElevationType}
Default
```

In this example, the anonymous user is not a split-token administrator, so the token has the default elevation type.

UI Access

One of the other security features introduced in Windows Vista is *User Interface Privilege Isolation (UIPI)*, which prevents a lower-privileged process from programmatically interacting with the user interface of a more privileged process. This is enforced using integrity levels, and it's another reason UAC administrators run at a `High` integrity level.

But UIPI presents a problem for applications that are designed to interact with the user interface, such as screen readers and touch keyboards. To get around this limitation without granting the process too much privilege, a token can set a UI access flag. Whether a process is granted UI access depends on the `UiAccess` setting in the executable's manifest file.

This UI access flag signals to the desktop environment that it should disable the UIPI checks. In [Listing 4-26](#), we query for this flag in a suitable process, the On-Screen Keyboard (OSK).

```
PS> $process = Start-Process "osk.exe" -PassThru
PS> $token = Get-NtToken -ProcessId $process.Id
PS> $token.UiAccess
True
```

Listing 4-26: Querying the UI access flag in the On-Screen Keyboard primary token

We start the OSK and open its `Token` object to query the UI access flag. To set this flag, the caller needs the `SeTcbPrivilege` privilege. The only way to create a UI access process as a normal user is to use the UAC service. Therefore, any UI access process needs to be started with `ShellExecute`, which is why we used `Start-Process` in [Listing 4-26](#). This all happens behind the scenes when you create the UI access application.

Virtualization

Another problem introduced in Vista because of UAC is the question of how to handle legacy applications, which expect to be able to write to administrator-only locations such as the *Windows* directory or the local machine registry hive. Vista implemented a special workaround: if a virtualization flag is enabled on the primary token, it will silently redirect writes from these locations to a per-user store. This made it seem to the process as if it had successfully added resources to secure locations.

By default, the virtualization flag is enabled on legacy applications automatically. However, you can specify it manually by setting a property on

the primary token. Run the commands in [Listing 4-27](#) in a non-administrator shell.

```
❶ PS> $file = New-NtFile -Win32Path C:\Windows\hello.txt -Access GenericWrite
New-NtFile : (0xC0000022) - {Access Denied}
A process has requested access to an object, but has not been granted those access rights.

PS> $token = Get-NtToken
❷ PS> $token.VirtualizationEnabled = $true
❸ PS> $file = New-NtFile -Win32Path C:\Windows\hello.txt -Access GenericWrite
❹ PS> $file.Win32PathName
C:\Users\user\AppData\Local\VirtualStore\Windows\hello.txt
```

Listing 4-27: Enabling virtualization on the Token object and creating a file in C:\Windows

In this listing, we first try to create a writable file, *C:\Windows\hello.txt* ❶. This operation fails with an access denied exception. We then get the current primary token and set the `VirtualizationEnabled` property to `True` ❷. When we repeat the file creation operation, it now succeeds ❸. If we query the location of the file, we find it's under the user's directory in a virtual store ❹. Only normal, unprivileged tokens can enable virtualization; system service and administrator tokens have virtualization disabled. You can learn whether virtualization is permitted by querying the `VirtualizationAllowed` property on the `Token` object.

Security Attributes

A token's *security attributes* are a list of name/value pairs that provide arbitrary data. There are three types of security attributes associated with a token: *local*, *user claims*, and *device claims*. Each security attribute can have one or more values, which must all be of the same type. [Table 4-5](#) shows the valid types for a security attribute.

Table 4-5: Security Attribute Types

Type name	Description
Int64	A signed 64-bit integer
UInt64	An unsigned 64-bit integer
String	A Unicode string
Fqbn	A fully qualified binary name; contains a version number and a Unicode string
Sid	A SID
Boolean	A true or false value, stored as an Int64, with 0 being false and 1 being true

Type name	Description
OctetString	An arbitrary array of bytes

A set of flags can be assigned to the security attribute to change aspects of its behavior, such as whether new tokens can inherit it. [Table 4-6](#) shows the defined flags.

Table 4-6: Security Attribute Flags

Flag name	Description
NonInheritable	The security attribute can't be inherited by a child process token.
CaseSensitive	If the security attribute contains a string value, the comparison should be case sensitive.
UseForDenyOnly	The security attribute is used only when checking for denied access.
DisabledByDefault	The security attribute is disabled by default.
Disabled	The security attribute is disabled.
Mandatory	The security attribute is mandatory.
Unique	The security attribute should be unique on the local system.
InheritOnce	The security attribute can be inherited once by a child, then should be set NonInheritable.

Almost every process token has the `TSA://ProcUnique` security attribute. This security attribute contains a unique LUID allocated during process creation. We can display its value for the effective token using `Show-NtTokenEffective`, as shown in [Listing 4-28](#).

```
PS> Show-NtTokenEffective -SecurityAttributes
SECURITY ATTRIBUTES
-----
Name           Flags           ValueType Values
-----
TSA://ProcUnique NonInheritable, Unique UInt64    {133, 1592482}
```

Listing 4-28: Querying the security attributes for the current process

From the output, we can see that the name of the attribute is `TSA://ProcUnique`. It has two `UInt64` values, which form a LUID when combined. Finally, it has two flags: `NonInheritable`, which means the security at-

tribute won't be passed to new process tokens, and `Unique`, which means the kernel shouldn't try to merge the security attribute with any other attribute on the system with the same name.

To set local security attributes, the caller needs to have the `SeTcb Privilege` privilege before calling `NtSetInformationToken`. User and device claims must be set during token creation, which we discuss in the next section.

Creating Tokens

Typically, LSASS creates tokens when a user authenticates to the computer. However, it can also create tokens for users that don't exist, such as virtual accounts used for services. These tokens might be interactive, for use in a console session, or they could be network tokens for use over the local network. A locally authenticated user can create another user's token by calling a Win32 API such as `LogonUser`, which calls into LSASS to perform the token creation.

We won't discuss LSASS at length until [Chapter 10](#). However, it's worth understanding how LSASS creates tokens. To do so, LSASS calls the `NtCreateToken` system call. As I mentioned earlier, this system call requires the `SeCreateTokenPrivilege` privilege, which is granted to a limited number of processes. This privilege is about as privileged as it gets, as you can use it to create arbitrary tokens with any group or user SID and access any resource on the local machine.

While you won't often have to call `NtCreateToken` from PowerShell, you can do so through the `New-NtToken` command so long as you have `SeCreateTokenPrivilege` enabled. The `NtCreateToken` system call takes the following parameters:

Token type Either primary or impersonation

Authentication ID The LUID authentication ID; can be set to any value you'd like

Expiration time Allows the token to expire after a set period

User The user SID

Groups The list of group SIDs

Privileges The list of privileges

Owner The owner SID

Primary group The primary group SID

Source The source information name

In addition, Windows 8 introduced the following new features to the system call, which you can access through the `NtCreateTokenEx` system call:

Device groups A list of additional SIDs for the device

Device claim attributes A list of security attributes to define device claims

User claim attributes A list of security attributes to define user claims

Mandatory policy A set of flags that indicate the token's mandatory integrity policy

Anything not in these two lists can be configured only by calling `NtSetInformationToken` after the new token has been created. Depending on what token property is being set, you might need a different privilege, such as `SeTcbPrivilege`. Let's demonstrate how to create a new token using the script in [Listing 4-29](#), which you must run as an administrator.

```
PS> Enable-NtTokenPrivilege SeDebugPrivilege
❶ PS> $imp = Use-NtObject($p = Get-NtProcess -Name lsass.exe) {
    Get-NtToken -Process $p -Duplicate
}
❷ PS> Enable-NtTokenPrivilege SeCreateTokenPrivilege -Token $imp
❸ PS> $token = Invoke-NtToken $imp {
    New-NtToken -User "S-1-0-0" -Group "S-1-1-0"
}
PS> Format-NtToken $token -User -Group
USER INFORMATION
-----
Name      Sid
----      --
❹ NULL SID S-1-0-0

GROUP SID INFORMATION
-----
Name      Attributes
----      -
❺ Everyone      Mandatory, EnabledByDefault, Enabled
Mandatory Label\System Mandatory Level Integrity, IntegrityEnabled
```

Listing 4-29: Creating a new token

A normal administrator does not have the `SeCreateTokenPrivilege` privilege by default. Therefore, we need to borrow a token from another process that does. In most cases, the easiest process to borrow from is LSASS. We open the LSASS process and its token, duplicating it to an impersonation token ❶. Next, we ensure that `SeCreateTokenPrivilege` is enabled on the token ❷. We can then impersonate the token and call `New-NtToken`, passing it a SID for the user and a single group ❸. Finally, we can print out the details for the new token, including its user SID set ❹ and group set ❺. The `New-NtToken` command also adds a default system integrity level SID that you can see in the group list.

Token Assignment

If a normal user account could assign arbitrary primary or impersonation tokens, it could elevate its privileges to access the resources of other users. This would be especially problematic when it comes to imperson-

ation, as another user account would need only open a named pipe to inadvertently allow the server to get an impersonation token.

For that reason, the SRM imposes limits on what a normal user can do without the `SeAssignPrimaryTokenPrivilege` and `SeImpersonationPrivilege` privileges. Let's take a look at the criteria that must be met to assign a token for a normal user.

Assigning a Primary Token

A new process can be assigned a primary token in one of three ways:

- It can inherit the token from the parent process.
- The token can be assigned during process creation (for example, using the `CreateProcessAsUser` API).
- The token can be set after process creation using `NtSetInformationProcess`, before the process starts.

Inheriting the token from the parent is by far the most common means of token assignment. For example, when you start an application from the Windows Start menu, the new process will inherit the token from the Explorer process.

If a process does not inherit a token from its parent, the process will be passed the token as a handle that must have the `AssignPrimary` access right. If the access to the `Token` object is granted, the SRM imposes further criteria on the token to prevent the assignment of a more privileged token (unless the caller's primary token has `SeAssignPrimaryTokenPrivilege` enabled).

The kernel function `SeIsTokenAssignableToProcess` imposes the token criteria. First it checks that the assigned token has an integrity level less than or equal to that of the current process's primary token. If that criterion is met, it then checks whether the token meets either of the criteria shown in **Figure 4-4**: namely, that the token is either a child of the caller's primary token or a sibling of the primary token.

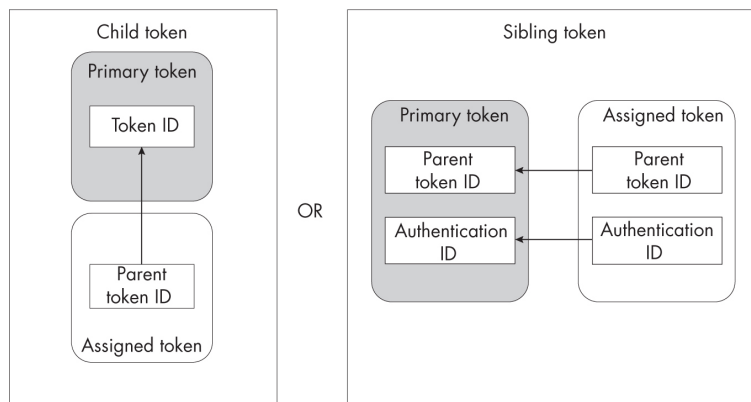


Figure 4-4: The `SeIsTokenAssignableToProcess` primary token assignment criteria

Let's first cover the case of a child token. A user process can create a new token based on an existing one. When this occurs, the `ParentTokenId` property in the new token's kernel object is set to the ID of the parent token. If

the new token's `ParentTokenId` matches the current primary token's ID value, then the assignment is granted. Restricted tokens are examples of child tokens; when you create a restricted token using `NtFilterToken`, the new token's parent token ID is set to the ID of the original token.

A *sibling token* is a token created as part of the same authentication session as the existing token. To test this criterion, the function compares the parent token ID and the authentication IDs of the two tokens. If they're the same, then the token can be assigned. This check also tests whether the authentication sessions are special sibling sessions set by the kernel (a rare configuration). Common examples of a sibling token include tokens duplicated from the current process token and lowbox tokens.

Note that the function doesn't check the user that the token represents, and if the token matches one of the criteria, it's possible to assign it to a new process. If it doesn't match the criteria, then the `STATUS_PRIVILEGE_NOT_HELD` error will be returned during token assignment.

How does the `runas` utility create a new process as a normal user with these restrictions? It uses the `CreateProcessWithLogon` API, which authenticates a user and starts the process from a system service that has the required privileges to bypass these checks.

If we try to assign a process token, we'll see how easily the operation can fail, even when we're assigning tokens for the same user. Run the code in [Listing 4-30](#) as a non-administrator user.

```
PS> $token = Get-NtToken -Filtered -Flags DisableMaxPrivileges
❶ PS> Use-NtObject($proc = New-Win32Process notepad -Token $token) {
    $proc | Out-Host
}
Process           : notepad.exe
Thread            : thread:11236 - process:9572
Pid               : 9572
Tid               : 11236
TerminateOnDispose : False
ExitStatus        : 259
ExitNtStatus      : STATUS_PENDING

❷ PS> $token = Get-NtToken -Filtered -Flags DisableMaxPrivileges -Token $token
PS> $proc = New-Win32Process notepad -Token $token
❸ Exception calling "CreateProcess" with "1" argument(s): "A required privilege
is not held by the client"
```

Listing 4-30: Creating a process using restricted tokens

Here, we create two restricted tokens and use them to create an instance of Notepad. In the first attempt, we create the token based on the current primary token ❶. The parent token ID field in the new token is set to the primary token's ID, and when we use the token during process creation, the operation succeeds.

In the second attempt, we create another token but base it on the one we created previously ❷. Creating a process with this token fails with a privi-

lege error ❸. This is because the second token's parent token ID is set to the ID of the crafted token, not the primary token. As the token doesn't meet either the child or sibling criterion, this operation will fail during assignment.

You can set the token after creating the process by using the `NtSetInformationProcess` system call or `ProcessAccessToken`, which PowerShell exposes with the `Set-NtToken` command (demonstrated in [Listing 4-31](#)).

```
PS> $proc = Get-NtProcess -Current
PS> $token = Get-NtToken -Duplicate -TokenType Primary
PS> Set-NtToken -Process $proc -Token $token
Set-NtToken : (0xC00000BB) - The request is not supported.
```

Listing 4-31: Setting an access token after a process has started

This assignment operation does not circumvent any of the assignment checks we've discussed. Once the process's initial thread starts executing, the option to set the primary token is disabled, so when we try to set the token on a started process we get the `STATUS_UNSUPPORTED` error.

Assigning an Impersonation Token

As with primary tokens, the SRM requires that an assigned impersonation token meet a specific set of criteria; otherwise, it will reject the assignment of the token to a thread. Interestingly, the criteria are not the same as those for the assignment of primary tokens. This can lead to situations in which it's possible to assign an impersonation token but not a primary token, and vice versa.

If the token is specified explicitly, then the handle must have the `Impersonate` access right. If the impersonation happens implicitly, then the kernel is already maintaining the token, and it requires no specific access right.

The `SeTokenCanImpersonate` function in the kernel handles the check for the impersonation criteria. As shown in [Figure 4-5](#), this check is significantly more complex than that for assigning primary tokens.

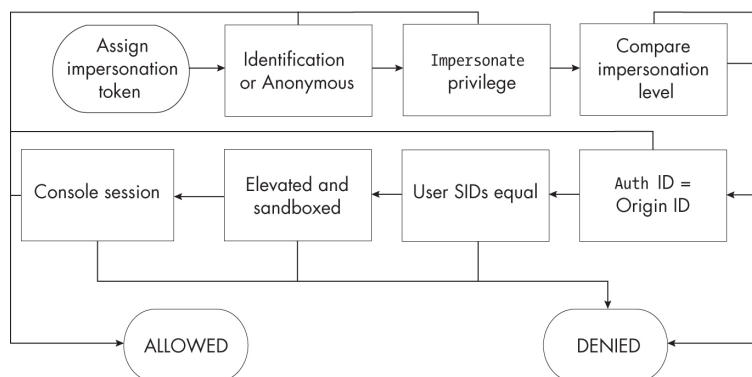


Figure 4-5: The `SeTokenCanImpersonate` impersonation token checks

Let's walk through each check and describe what it considers on both the impersonation and the primary token. Note that, because it's possible to assign an impersonation token to a thread in another process (if you have an appropriate handle to that thread), the primary token being checked is the one assigned to the process that encapsulates the thread, and not the primary token of the calling thread. The function performs the following verification steps:

1. Check for an Identification or Anonymous impersonation level. If the impersonation token has one of these levels, assigning it to the thread isn't a security risk, and the SRM immediately allows the assignment. This check also allows assignment if the impersonation token represents the anonymous user based on its authentication ID.
2. Check for the impersonate privilege. If `SeImpersonatePrivilege` is enabled, the SRM again immediately allows the assignment.
3. Compare integrity levels of the primary and impersonation tokens. If the primary token's integrity level is less than that of the impersonation token, the assignment is denied. If it's the same or greater, the checks continue.
4. Check that the authentication ID equals the origin ID. If the origin logon identifier of the impersonation token equals the authentication identifier of the primary token, the SRM allows the assignment. Otherwise, it continues making checks.

Note that this check has an interesting consequence. As discussed earlier in this chapter, the origin logon identifier of normal user tokens is set to the authentication identifier of the *SYSTEM* user. This is because the authenticating process runs as the *SYSTEM* user. Therefore, the *SYSTEM* user can impersonate any other token on the system if it meets the integrity level requirement, even if the `SeImpersonatePrivilege` privilege is not enabled.

5. Check that the user SIDs are equal. If the primary token's user SID does not equal the impersonation token's user SID, the SRM denies the assignment. Otherwise, it continues making checks. This criterion allows a user to impersonate their own user account but blocks them from impersonating another user unless they have the other user's credentials. When authenticating the other user, LSASS returns an impersonation token with the origin logon identifier set to the caller's authentication identifier, so the token will pass the previous check and the user SIDs will never be compared.

6. Check for the `Elevated` flag. This check ensures that the caller can't impersonate a more privileged token for the same user. If the impersonation token has the `Elevated` flag set but the primary token does not, the impersonation will be denied. Versions of Windows prior to 10 did not perform this check, so previously it was possible to impersonate a UAC administrator token if you first reduced the integrity level.

7. Check for sandboxing. This check ensures that the caller can't impersonate a less-sandboxed token. To impersonate a lowbox token, the new token must either match the package SID or be a restricted package SID of the primary token; otherwise, impersonation will be denied. No check is made on the list of capabilities. For a restricted token, it's enough that the new token is also a restricted token, even if the list of restricted SIDs is different. The same applies to write-restricted tokens. The SRM has various hardening mechanisms to make it difficult to get hold of a more privileged sandbox token.

8. Check the console session. This final step checks whether the console session is session 0 or not. This prevents a user from impersonating a token in session 0, which can grant elevated privileges (such as being able to create global `Section` objects).

You might assume that if the function denies the assignment it will return a `STATUS_PRIVILEGE_NOT_HELD` error, but that is not the case. Instead, the SRM duplicates the impersonation token as an Identification-level token and assigns it. This means that even if the impersonation assignment fails, the thread can still inspect the properties of the token.

You can check whether you can impersonate a token using the `Test - NtTokenImpersonation` PowerShell command. This command impersonates the token and reopens it from the thread. It then compares the impersonation level of the original token and the reopened token and returns a Boolean result. In [Listing 4-32](#), we run through a simple example that would fall foul of the integrity level check. Note that it's best not to run this script in a PowerShell process you care about, as you won't be able to restore the original integrity level.

```
PS> $token = Get-NtToken -Duplicate
PS> Test-NtTokenImpersonation $token
True

PS> Set-NtTokenIntegrityLevel -IntegrityLevel Low
PS> Test-NtTokenImpersonation $token
False

PS> Test-NtTokenImpersonation $token -ImpersonationLevel Identification
True
```

Listing 4-32: Checking token impersonation

These checks are quite simple. First we get a duplicate of the current process token and pass it to `Test-NtTokenImpersonation`. The result is `True`, indicating that we could impersonate the token at Impersonation level. For the next check, we lower the integrity level of the current process's primary token to `Low` and run the test again. This time it returns `False`, as it's no longer possible to impersonate the token at the Impersonation level. Finally, we check if we can impersonate the token at the Identification level, which also returns `True`.

Worked Examples

Let's walk through some worked examples so you can see how to use the various commands presented in this chapter for security research or systems analysis.

Finding UI Access Processes

It's sometimes useful to enumerate all the processes you can access and check the properties of their primary tokens. This can help you find processes running as specific users or with certain properties. For example, you could identify processes with the UI access flag set. Earlier in this chapter, we discussed how to check the UI access flag in isolation. In [Listing 4-33](#), we'll perform the check for all processes we can access.

```
PS> $ps = Get-NtProcess -Access QueryLimitedInformation -FilterScript {
    Use-NtObject($token = Get-NtToken -Process $_ -Access Query) {
        $token.UIAccess
    }
}
PS> $ps
Handle Name      NtTypeName Inherit ProtectFromClose
-----
3120 ctfmon.exe Process False False
3740 TabTip.exe Process False False

PS> $ps.Close()
```

Listing 4-33: Finding processes with UI access

We start by calling the `Get-NtProcess` command to open all processes with `QueryLimitedInformation` access. We also provide a filter script. If the script returns `True`, the command will return the process; otherwise, it will close the handle to the process.

In the script, we open the process's token for `Query` access and return the `UIAccess` property. The result filters the process list to only processes running with UI access tokens. We display the processes we've found.

Finding Token Handles to Impersonate

There are several official ways of getting access to a token to impersonate, such as using a remote procedure call or opening the process's primary token. Another approach is to find existing handles to `Token` objects that you can duplicate and use for impersonation.

This technique can be useful if you're running as a non-administrator user with the `SeImpersonatePrivilege` privilege (as in the case of a service account such as `LOCAL SERVICE`), or to evaluate the security of a sandbox to make sure the sandbox can't open and impersonate a more privileged token. You can also use this technique to access another user's resources by waiting for them to connect to the Windows machine, such as over the network. If you grab the user's token, you can reuse their identity without

needing to know their password. [Listing 4-34](#) shows a simple implementation of this idea.

```
PS> function Get-ImpersonationTokens {
    ❶ $hs = Get-NtHandle -ObjectType Token
    foreach($h in $hs) {
        try {
            ❷ Use-NtObject($token = Copy-NtObject -Handle $h) {
                ❸ if (Test-NtTokenImpersonation -Token $token) {
                    Copy-NtObject -Object $token
                }
            } catch {
            }
        }
    }
}
❹ PS> $tokens = Get-ImpersonationTokens
❺ PS> $tokens | Where-Object Elevated
```

Listing 4-34: Finding elevated Token handles to impersonate

In the `Get-ImpersonationTokens` function, we get a list of all handles of type `Token` using the `Get-NtHandle` command ❶. Then, for each handle, we try to duplicate the handle to the current process using the `Copy-NtObject` command ❷. If this succeeds, we test whether we can successfully impersonate the token; if so, we make another copy of the token so it doesn't get closed ❸.

Running the `Get-ImpersonationTokens` function returns all accessible token handles that can be impersonated ❹. With these `Token` objects, we can query for properties of interest. For example, we can check whether the token is elevated or not ❺, which might indicate that we could use the token to gain additional privileged groups through impersonation.

Removing Administrator Privileges

One thing you might want to do while running a program as an administrator is temporarily drop your privileges so that you can perform some operation without damaging the computer, such as accidentally deleting system files. To perform the operation, you can use the same approach that UAC uses to create a filtered, lower-privileged token. Run the code in [Listing 4-35](#) as an administrator.

```
PS> $token = Get-NtToken -Filtered -Flags LuaToken
PS> Set-NtTokenIntegrityLevel Medium -Token $token
PS> $token.Elevated
False

PS> "Admin" > "$env:windir\admin.txt"
PS> Invoke-NtToken $token {"User" > "$env:windir\user.txt"}
out-file : Access to the path 'C:\WINDOWS\user.txt' is denied.

PS> $token.Close()
```

Listing 4-35: Removing administrator privileges

We start by filtering the current token and specifying the `LuaToken` flag. This flag removes all administrator groups and the additional privileges that a limited token is not allowed to have. The `LuaToken` flag does not lower the integrity level of the token, so we must set it to `Medium` manually. We can verify the token is no longer considered an administrator by checking that the `Elevated` property is `False`.

To see the effect in action, we can now write a file to an administrator-only location, such as the *Windows* directory. When we try this using the current process token, the operation succeeds. However, when we try to perform the operation while impersonating the token, it fails with an access denied error. You could also use the token with the `New-Win32Process` PowerShell command to start a new process with the lower-privileged token.

Wrapping Up

This chapter introduced the two main types of tokens: primary tokens, which are associated with a process, and impersonation tokens, which are associated with a thread and allow a process to temporarily impersonate a different user. We looked at the important properties of both types of tokens, such as groups, privileges, and integrity levels, and how those properties affect the security identity that the token exposes. We then discussed the two types of sandbox tokens (restricted and lowbox), which applications such as web browsers and document readers use to limit the damage of a potential remote code execution exploit.

Next, we considered how tokens are used to represent administrator privilege, including how Windows implements User Account Control and split-token administrators for normal desktop users. As part of this discussion, we explored the specifics of what the operating system considers to be an administrator or elevated token.

Finally, we discussed the steps involved in assigning tokens to processes and threads. We defined the specific criteria that need to be met for a normal user to assign a token and how the checks for primary tokens and impersonation tokens differ.

In the next chapter we're going to discuss security descriptors. These define what access will be granted to a resource based on the identity and groups present in the caller's access token.